

Machine Learning for Systems and Control

Policy Learning for an Unbalanced Disc

Nicu Gurmuzachi
Student ID: 2372800

Chris Hogendoorn
Student ID: 1602403

Zijian Dong
Student ID: 2268264

Abstract—This report presents the modelling and control design for an unbalanced disc system. The system dynamics are identified using Gaussian Process and Artificial Neural Network models. Reinforcement learning methods are then applied to learn swing-up and stabilization policies, followed by hardware testing on the physical setup.

Index Terms—System Identification, Gaussian Process, Neural Network, Reinforcement Learning, Q-learning, Actor-Critic, Unbalanced Disc

I. INTRODUCTION

The unbalanced disc is a disc carrying an off-centre mass, mounted on the shaft of a DC motor, which mechanically behaves as a pendulum. Gravity acts on the off-centre mass to produce a position-dependent restoring torque, while the motor voltage provides the actuation. The system has a stable equilibrium at the bottom and an unstable one at the upright position, and because the input voltage is limited the disc cannot be driven straight to the top, which makes the control problem nontrivial.

The goal of this project is: First, identify the dynamics of the disc from measured data using both Gaussian Process and artificial neural network models, evaluated on one-step-ahead prediction and free-run simulation; Second, a control policy is learned through reinforcement learning that swings the disc up from the bottom position, stabilises it at the upright position, and finally tracks a reference around that position.

II. SYSTEM DESCRIPTION

A. Unbalanced Disc Dynamics

The setup behaves as an unbalanced disc mounted on the shaft of a DC motor, which mechanically acts as a pendulum. The disc carries an off-centre mass, so gravity produces a restoring torque that depends on the angular position, and the motor voltage provides the actuation. The system is described by two states: the angle θ of the disc, measured in radians clockwise from the stable bottom position, and the angular velocity $\omega = \dot{\theta}$, in radians per second. The control input is the voltage u applied to the motor, in volts. Both θ and ω are available from the setup, although for the identification task only the voltage and the measured angle are used.

The motion is defined by

$$\begin{aligned}\dot{\theta}(t) &= \omega(t), \\ \dot{\omega}(t) &= -\omega_0^2 \sin(\theta(t)) - \gamma \omega(t) - F_c \text{sign}(\omega(t)) + K_u u(t),\end{aligned}\tag{1}$$

where ω_0 is the base frequency of the pendulum, γ the viscous friction coefficient, F_c the Coulomb friction coefficient and K_u the input gain. The $\sin(\theta)$ term makes the dynamics nonlinear, the $\text{sign}(\omega(t))$ to a lesser degree, and gives the system its two natural equilibria: a stable one at the bottom ($\theta = 0$) and an unstable one at the upright position ($\theta = \pi$), which is the target the controller must reach and hold.

The applied voltage is limited in software to

$$u(t) \in [-3, +3] \text{ V}.\tag{2}$$

This saturation is not merely a safety bound: it makes the system *underactuated*, in the sense that the available torque is not large enough to drive the disc directly from the bottom to the upright position in a single motion. A successful controller must instead build up energy by swinging the disc back and forth before bringing it to the top, which is what makes both the modelling and the control problem nontrivial.

B. Available Data and Simulator

To support the system identification and control tasks, a set of benchmark datasets and a software simulator are provided.

The datasets are used primarily for the system identification task and are supplied in various formats (CSV, MAT, and NPZ). They consist of three main parts:

- **Training and test data:** This dataset contains comprehensive historical sequences of applied motor voltages and the corresponding measured disk angles. It can be freely partitioned into training, validation, and test sets to train and optimize the prediction models.
- **Hidden test prediction data:** Designed to evaluate one-step-ahead prediction performance, this dataset contains past input and output sequences where specific future output values are omitted (set to zero) and must be estimated by the learned model.
- **Hidden test simulation data:** Used for evaluating free-run simulation capabilities, this dataset provides the full input sequence but only the first 50 measured output values. The model must simulate the remaining output trajectory based entirely on its learned dynamics.

In addition to the static datasets, a software simulator is provided in Python, implemented as a custom *Gymnasium* environment. The simulator provides a nominal model of the disc dynamics and was used as the main training environment for the reinforcement learning controllers. It operates at a fixed sampling interval (e.g., $\Delta t = 0.025$ s) and strictly enforces the actuation limits ($u(t) \in [-3, +3]$ V). This environment

is essential for the control task, allowing the reinforcement learning agent to interact with the system, receive state observations and rewards, and iteratively learn the swing-up and stabilization policies.

III. SYSTEM IDENTIFICATION

A. Data Processing

The model is built from the input voltage u and the measured angle θ only; the angular velocity ω , although available from the setup, is not used, in accordance with the assignment. Each past angle entering the Nonlinear AutoRegressive model with eXogenous input (NARX) regressor is encoded as $(\sin \theta, \cos \theta)$ to stay continuous across the $\pm\pi$ wrap-around, and the regressor columns are standardised to zero mean and unit variance before training. The data is partitioned with train/test split.

B. Model Structures

A NARX parametrisation was chosen for GP over a state-space or NOE structure because it keeps the regressor explicit and lets the GP operate directly on measured lagged inputs and outputs, which is well suited to the dataset size and to the sparse approximation discussed below.

C. Gaussian Process Model

The disc dynamics are identified with a Gaussian Process (GP) in a NARX input-output structure. The one-step predictor is

$$\theta(k) = f(u(k-1), \dots, u(k-n_b), \theta(k-1), \dots, \theta(k-n_a)) + \varepsilon(k), \quad (3)$$

where the unknown map f is given a GP prior and ε is a noise term.

A point that matters for this particular system is the angle wrap-around at $\pm\pi$. Feeding the raw angle into a smooth kernel introduces an artificial discontinuity exactly in the region the swing-up controller later has to pass through. Each past angle is therefore encoded as the pair $(\sin \theta, \cos \theta)$, so that the regressor is continuous across the wrap.

For the covariance function, a squared-exponential (RBF) kernel with Automatic Relevance Determination is used, giving one length-scale per input dimension, together with a white-noise term for the measurement noise. The ARD structure is informative in its own right: after training, inputs with a large length-scale contribute little, which gives a direct read on how many lags the data actually supports and complements the explicit n_a, n_b search. The inputs are standardised before training so that the length-scales start on a comparable scale.

The training set contains $N \approx 28,000$ samples. Exact GP inference is impractical at this size: training requires inverting an $N \times N$ covariance matrix, so the cost grows with N^3 and the matrix alone becomes too large to store comfortably. A sparse GP with inducing points (FITC) is therefore used. This reduces the cost to $\mathcal{O}(NM^2)$ for M inducing points, while still using all N training samples to place those points, so no data is discarded. The number of inducing points M is

treated as a hyper-parameter, since it trades fit quality against computation. As shown below, its effect is small for one-step prediction but visible for free-run simulation.

Given the model structure, the kernel hyper-parameters, the noise variance and the inducing-point locations are obtained by maximising the marginal likelihood of the training data, which is the standard evidence-based criterion for GP models and automatically balances data fit against model complexity. The optimisation is gradient-based. Because the marginal likelihood is non-convex and can exhibit multiple local optima, the optimisation can be repeated from several random initialisations and the best solution retained; this was kept limited here to contain the training time on the full dataset.

The model is evaluated on the two tasks required by the assignment: prediction and simulation. In *prediction* the true past outputs are supplied at every step, so the model predicts only one step ahead and errors do not accumulate. In *simulation* the model is run in free-run: its own past predictions are fed back as inputs, so any error at one step propagates into all later steps. Simulation is thus the stricter test of whether the underlying dynamics, rather than only the short-term correlation between consecutive samples, have been captured. The two tasks do not necessarily share the same optimal hyper-parameters: the configuration that minimises the prediction error need not be the one that minimises the simulation error. Since the same identified model is used for both tasks, a single configuration must be chosen. Each task is therefore swept separately, and the configuration that performs well on both is selected, rather than picking the optimum of one task at the expense of the other.

The hyper-parameters n_a, n_b (swept with $n_a = n_b$) and M were tuned on a held-out test split, following the system identification cycle, by sweeping each task separately. Tables I and II report the prediction and simulation RMSE across the grid. For prediction the lag order is the dominant factor while M has almost no effect; for simulation the lag order again dominates, with $M = 200$ outperforming $M = 500$ at every lag. A plausible explanation is that a larger inducing set fits the one-step target slightly more aggressively, which makes the free-run feedback loop marginally less stable. Both tasks reach their best accuracy at $n_a = n_b = 4$. The prediction-optimal row ($n_a = n_b = 5, M = 500$) improves on $n_a = n_b = 4, M = 200$ by less than 0.0002 rad, which is within the run-to-run variation caused by the random selection of inducing points, while costing roughly twenty percent on the simulation error. The configuration $n_a = n_b = 4, M = 200$ is therefore selected for both tasks, as it is the simulation optimum and is statistically indistinguishable from the best prediction result.

As shown in Table III, the selected model beats the linear baseline on both tasks and surpasses the reference good-model figure for prediction. For simulation it improves on the linear baseline by roughly a factor of six and reduces the untuned error (0.067 rad at $n_a = n_b = 5$) by about a third, though it does not yet reach the reference good-model figure. As anticipated, the simulation error remains larger than the

prediction error, reflecting the accumulation of error in free-run.

TABLE I
PREDICTION (ONE-STEP-AHEAD) RMSE OVER THE HYPER-PARAMETER GRID.

$n_a = n_b$	M	RMSE [rad]	RMSE [deg]
2	200	0.00418	0.239
2	500	0.00419	0.240
3	200	0.00356	0.204
3	500	0.00363	0.208
4	200	0.00340	0.195
4	500	0.00333	0.191
5	200	0.00358	0.205
5	500	0.00327	0.187

TABLE II
SIMULATION (FREE-RUN) RMSE OVER THE HYPER-PARAMETER GRID.

$n_a = n_b$	M	RMSE [rad]	RMSE [deg]
2	200	0.07564	4.334
2	500	0.09188	5.264
3	200	0.05093	2.918
3	500	0.05254	3.010
4	200	0.04220	2.418
4	500	0.04512	2.585
5	200	0.04601	2.636
5	500	0.05056	2.897

TABLE III
FINAL GP TEST ERRORS FOR THE SELECTED CONFIGURATION
($n_a = n_b = 4$, $M = 200$), COMPARED WITH THE PROVIDED REFERENCE FIGURES.

	RMSE [rad]	RMSE [deg]
<i>Prediction</i>		
GP (this work)	0.00340	0.195
Good model (ref.)	0.00382	0.219
Linear (ref.)	0.00665	0.381
<i>Simulation</i>		
GP (this work)	0.04220	2.418
Good model (ref.)	0.02710	1.550
Linear (ref.)	0.25500	14.600
Lower bound (ref.)	0.01950	1.120

D. Artificial Neural Network Model

For the ANN approach, a feedforward neural network was implemented as a nonlinear autoregressive model with exogenous input (NARX). The purpose of this model is to learn a one-step-ahead input-output approximation of the disc dynamics directly from measured data. The model uses a fixed window of past motor voltages and past measured angles as the regressor,

$$\phi_k = [u_{k-1}, \dots, u_{k-n_b}, \theta_{k-1}, \dots, \theta_{k-n_a}],$$

where n_a and n_b denote the output and input history lengths. In our implementation, both were set to 15 samples.

Instead of directly predicting the absolute next angle, the network was trained to predict the angle increment.

$$\Delta\theta_k = \theta_k - \theta_{k-1}.$$

The final angle prediction was then reconstructed as

$$\hat{\theta}_k = \theta_{k-1} + \widehat{\Delta\theta}_k.$$

This formulation was used because the angle changes smoothly between consecutive samples, making the learning target smaller and easier to model than the absolute angle.

Two feedforward ANN configurations were evaluated. The first used the raw lagged input-output vector directly. The second used an augmented feature representation including engineered trigonometric angle features, which help represent the periodic nature of the disk angle. The advanced network used three hidden layers with 128, 128, and 64 neurons, respectively. SiLU activation functions and layer normalization were used after the hidden layers to improve training stability. The network output layer was linear, producing a scalar prediction of $\Delta\theta_k$.

1) *Training Procedure and Validation:* The available dataset was split chronologically into training, validation, and test segments. The first 25,000 samples were used for training, the next 5,000 samples for validation, and the remaining samples for internal testing. A chronological split was used instead of random shuffling in order to preserve the time-series structure of the data and to evaluate the model on future samples not seen during training.

Both the input features and the target angle increments were normalized using standard scaling. The input scaler was fitted on the training data and then applied to the validation, test, and hidden-test data. The target scaler was similarly fitted on the training angle increments and inverted after prediction to recover the physical angle increment in radians.

The networks were trained using the AdamW optimizer with mean squared error loss. Weight decay was included as regularization to reduce overfitting. Early stopping was applied based on the validation loss: the model parameters with the lowest validation loss were stored, and training was stopped when the validation loss did not improve for 25 consecutive epochs.

The advanced ANN achieved the best prediction result among the evaluated feedforward configurations, with an RMS prediction error of 0.00311 rad, corresponding to 0.178°, and an NRMS of 0.602%. In free-run simulation, where prediction errors accumulate recursively, the RMS error increased to 0.0401 rad, corresponding to 2.30°.

E. Model Evaluation

The values for the GP-NARX model are taken from the selected configuration reported in the GP section. The NARX ANN achieved the lowest one-step prediction RMSE and a slightly lower free-run simulation RMSE, while both nonlinear models substantially outperformed the linear baseline.

TABLE IV
SYSTEM IDENTIFICATION RESULTS

Model	Prediction RMSE [rad]	Simulation RMSE [rad]
Linear baseline	0.00665	0.25500
GP-NARX	0.00340	0.04220
NARX ANN	0.00311	0.04015

IV. REINFORCEMENT LEARNING FOR SWING-UP

A. RL Problem Formulation

To apply reinforcement learning to the unbalanced disc, the control task is framed as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. We map the physical system described in Section 2 to this MDP as follows:

- **State Space ($\mathcal{S}$):** The physical system's state is fully described by its angle θ and angular velocity ω . However, feeding the raw angle directly to a learning agent introduces a harsh numerical discontinuity when the disc wraps around at $\theta = \pm\pi$. To provide a smooth state representation, both of our RL implementations map the angle to its continuous trigonometric components, $\cos(\theta)$ and $\sin(\theta)$. Depending on the specific algorithm, additional features or normalization scaling are applied to formulate the final observation vector.
- **Action Space ($\mathcal{A}$):** The underlying action space is the motor voltage bounded by hardware limits: $u(t) \in [-3, +3]$ V. This continuous boundary is handled differently depending on the algorithm: our Q-Learning approach discretizes this space into a specific set of finite actions for precise micro-adjustments, whereas our Actor-Critic approach utilizes the continuous action space directly.
- **Reward Function ($\mathcal{R}$):** Because the system is underactuated, a sparse reward (e.g., +1 only when balanced at the top) is extremely difficult to learn. The reward signal must be carefully shaped to guide the agent to pump kinetic energy into the system, while simultaneously penalizing position errors and excessive speeds that lead to overshooting the unstable equilibrium.
- **Transition Constraints & Sim-to-Real Gap ($\mathcal{P}$):** Beyond the ideal dynamics defined in Eq. 1, the MDP transitions are severely constrained by physical hardware realities. The experimental setup possesses an inherent input delay of approximately 2 timesteps, and the ω sensor exhibits significant noise and a wandering bias. Successful policies must be robust enough to handle these unmodeled delays and observation errors when transferred from simulation to the physical hardware.

B. Q-learning Based Method

1) *Architecture:* We used Radial Basis Functions (RBF) Q-Learning for the Q-learning part. We chose RBF Q-Learning over vanilla tabular Q-Learning since it naturally handles continuous state spaces. We avoided Deep Q-Learning (DQN) because its high sensitivity to hyperparameters and massive

computational cost were prohibitive given our hardware and time constraints. RBF Q-Learning offered a much smaller hyperparameter search space, allowing us to discover reliable policies much more efficiently.

When using RBF Q-Learning, the action space still needs to be discretized. We used the following input voltages, providing finer-grained control near zero to allow for delicate balancing adjustments:

$$A = \{-3.0, -2.0, -1.0, -0.5, -0.2, -0.1, -0.05, 0.0, 0.05, 0.1, 0.2, 0.5\} \quad (4)$$

2) *Reward function:* We started the reward function off with two primary terms: the position reward and the energy reward.

We wanted the agent to get as close as possible to the top. For the state, we used $\cos(\theta)$, $\sin(\theta)$, and ω . When θ is $\pm 180^\circ$ (the top position), $\cos(\theta) = -1$. Thus, we set the position reward as:

$$R_{pos} = -\cos(\theta) \quad (5)$$

This penalized positions in the lower half of the circle and rewarded positions in the upper half, increasing steadily towards the top.

Since we also wanted the agent to learn a swing-up, we implemented the energy reward. The total energy $E_{current}$ is the sum of the kinetic and potential energy:

$$E_{current} = \frac{1}{2}\omega^2 + (R_{pos} \cdot 100) \quad (6)$$

The position multiplier of 100 is a critical scaling factor. A typical physical swing generates kinetic energies ($\frac{1}{2}\omega^2$) in the range of 0 to 150. If the position reward R_{pos} (which only ranges from -1 to 1) was not scaled up, the kinetic energy would mathematically overpower the position in the calculation. By multiplying R_{pos} by 100, we ensure the potential and kinetic energies are of the exact same numerical magnitude, properly balancing the trade-off between speed and height.

The target energy E_{target} was set to 100, because the potential energy at the top (with zero kinetic energy) is exactly 100. The energy penalty was defined as:

$$R_{energy} = -\frac{|E_{current} - E_{target}|}{100} \quad (7)$$

Each of these terms was multiplied by a weight to reach the final reward value:

$$\text{Reward} = w_{pos}R_{pos} + w_{energy}R_{energy} \quad (8)$$

We then ran a hyperparameter sweep across the following 7 parameters: n_{bins} , σ , α , γ , ϵ_{decay} , w_{energy} , and $w_{position}$. The value ranges for the tuning sweep can be found in the appendix VI.

3) *Tuning:* In the tuning stage, we trained each trial for 400 episodes. We pruned a run at episode 200 if its current reward was worse than the median of all previous trials, this was done by optuna's MedianPruner function. To see how well the model performed, we paused the training and ran the policy for 5 evaluation episodes. We averaged the value of $-\cos(\theta)$ over these 5 runs to obtain the Optuna score, which was used to rank the hyperparameters. We ran 100 trials.

4) *v0 models*: After the tuning, we chose the 5 best parameter sets and trained 5 models with these parameters for 800 episodes. We call these the v0 models. These were then evaluated on the hardware setup.

When we evaluated them on the physical setup, we found that they were all attempting to balance but when they fell then they would start spinning the disk fast in one direction for the remainder of the episode. We call this phenomenon a death spiral. We noticed that the ω sensor was reading out a wild range of values when at rest. We believe that the ω sensor had a wandering bias. To fix this, we applied an `AdaptiveBiasWrapper` which derived the expected ω from the θ observations. We applied an exponential moving average filter to filter out the bias without inheriting the noise from the ω sensor. It successfully ignores sudden noisy spikes.

- **Reward function**: $w_{pos}R_{pos} + w_{energy}R_{energy}$
- **Optuna scoring**: $\frac{1}{5} \sum (-\cos(\theta))$

5) *v1 models*: We retrained the v0 models using this new adaptive bias filter to create the v1 models. When we tested these on the hardware, the drift was fixed, but the models were still too aggressive and were overshooting into death spirals. Thus, we added a stability penalty (P_{stab}) to the reward function. This penalizes having high angular velocities while near the top to force the agent to brake:

$$P_{stab} = \begin{cases} w_{pos} \cdot (0.1\omega^2 + 0.5a^2) & \text{if } \cos(\theta) < -0.8 \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

$$\text{Reward} = w_{pos}R_{pos} + w_{energy}R_{energy} - w_{stab}P_{stab} \quad (10)$$

Because we changed the reward logic and added a new stability penalty (which required a new weight), we had to run the hyperparameter tuning again. To force Optuna to find smooth policies, we also changed the evaluation score to include a smoothness penalty based on the Total Variation of the motor voltage:

- **Optuna scoring**: $\sum (-\cos(\theta)) - 0.5 \sum |\Delta V|$

6) *v2 models*: To fully resolve the hardware failures, we fundamentally overhauled our evaluation strategy by introducing an entirely new "Robust Evaluation" method. We tested the model using 5 environmental variants: nominal, low friction, high friction, weak motor, and strong motor. The Optuna score was optimized for the mean minus 0.5 standard deviations across all variants.

The score for a single variant episode was drastically upgraded to accurately capture desirable behavior:

$$\text{Score} = B_{pos} + B_{catch} + B_{balance} + B_{streak} - P_{speed} - P_{unsafe} - P_{balancing_weight} \quad (11)$$

- **Position Bonus**: $-\cos(\theta)$
- **Catch Bonus**: $+2.0$ if $\cos(\theta) < -0.8$ and $|\omega| < 8.0$
- **Balance Bonus**: $+6.0$ if $\cos(\theta) < -0.8$ and $|\omega| < 4.0$
- **Streak Bonus**: $+4.0 \times$ (longest continuous balance streak)
- **Speed Penalty**: $-0.015 \sum \omega^2$ (penalizes swinging unnecessarily fast)

- **Unsafe Penalty**: -75.0 if peak speed ever exceeds 38.0 rad/s
- **Chatter Penalty**: $-0.75 \sum |\Delta V|$ (penalizes jittery motor commands)
- **Miss Penalty**: $-20.0 \times$ (Mean of the 40 best positions) (harshly penalizes policies that never manage to get close to the top)

Simultaneously, the training reward function was upgraded to explicitly reward active balancing at the top by introducing $B_{balance}$. The P_{stab} coefficient for ω was also subtly tweaked from 0.1 to 0.15:

$$B_{balance} = \begin{cases} w_{pos} \cdot \left(1.0 - \min\left(1.0, \frac{|\omega|}{6.0}\right)\right) & \text{if } \cos(\theta) < -0.8 \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

$$\text{Reward} = w_{pos}R_{pos} + w_{energy}R_{energy} + w_{balance}B_{balance} - w_{stab}P_{stab} \quad (13)$$

After establishing this robust new evaluation framework, we ran an initial exploratory hyperparameter sweep using the wide parameter bounds defined in Table VI. To hone in on the absolute best configurations, we extracted the top 10% of successful trials from this initial database and analyzed their parameter distributions.

The analysis revealed that the best-performing policies tightly clustered around specific agent architecture parameters. Based on these insights, we aggressively narrowed the Optuna search space for our final tuning sweep: the number of RBF bins (n_{bins}) was locked permanently to 10, the RBF width (σ) was narrowed to $[0.20, 0.31]$, and the learning rate (α) was clamped to $[0.010, 0.040]$. The reward function weights were left at their original wide bounds to allow Optuna to freely optimize the behavior given this focused architecture.

We ran 250 tuning trials using these focused ranges. When we found out we could run multiple trials at the same time by using multiprocessing we could run a lot more trials in the same time, this is why the number of trials was increased by a lot. We trained 5 models on the 5 best parameters obtained from this final tuning. These are the v2 models. We tested these on the hardware, and this is where we finally saw some good behavior. The 4th model managed to balance the full episode in 1/6 tests (See Figure 1), though this was at a slight offset angle, and balanced for half of the episode in 2/6 tests (See Figure 2). The 5th model also managed to balance perfectly once (See Figure 3).

The parameters of these models are in Table V. As we can see, both models took very different approaches regarding the reward weights. The 4th model heavily focused on the balancing weight ($w_{balance}$) and subsequently didn't need a high stability penalty. The 5th model heavily prioritized the position weight and almost entirely ignored the energy penalty.

7) *v3 models*: Despite the successes, we figured out that when the setup went into a death spiral, the agent wasn't able to get out of it. We reevaluated our reward function again and noticed that the agent wasn't being properly penalized for having too much energy. In fact, it had learned to "farm" the energy penalty by sitting at the bottom and spinning violently.

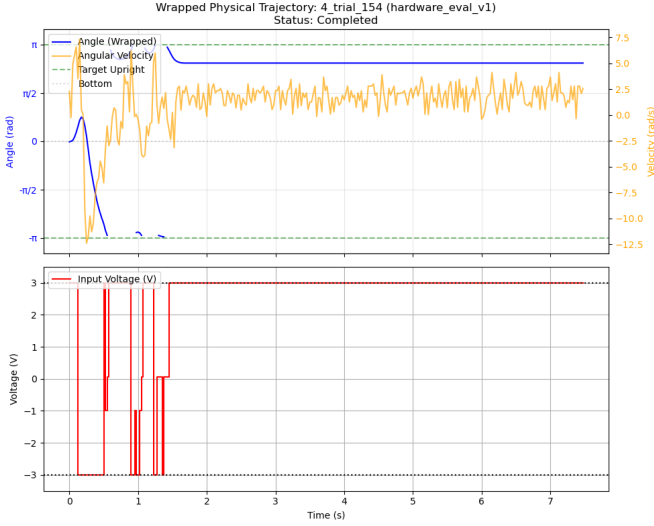


Fig. 1. First hardware test of model 4 of batch v2

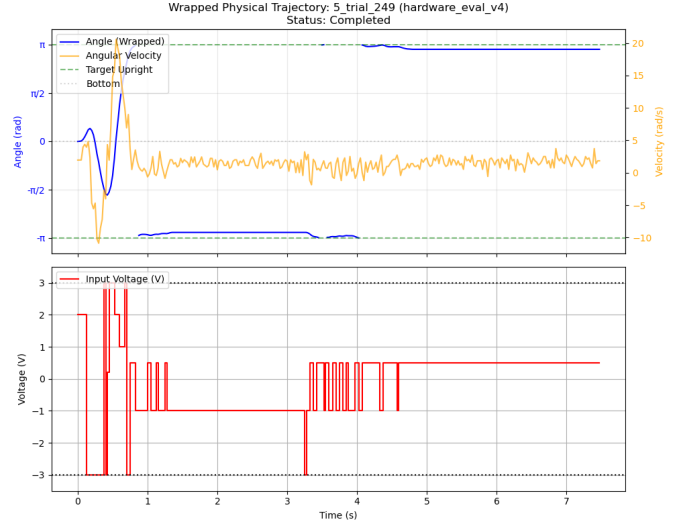


Fig. 3. First hardware test of model 5 of batch v2

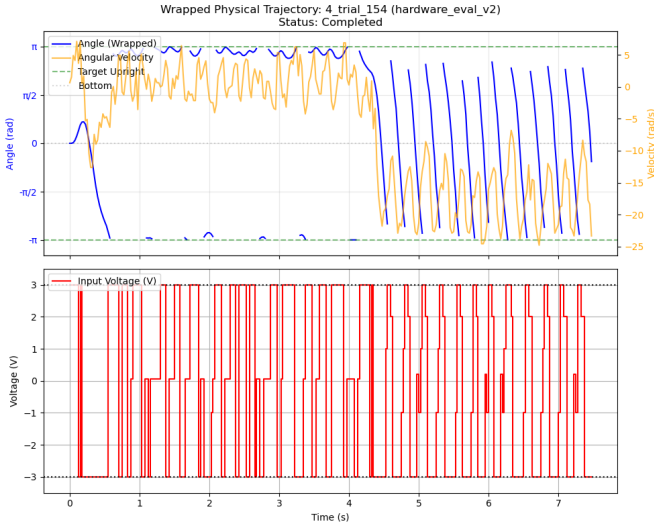


Fig. 2. Second hardware test of model 4 of batch v2

To fix this, we added a strict excess energy penalty (P_{excess}) to the reward formula. We applied a 10% tolerance margin around the target energy before aggressively squaring the penalty:

$$E_{excess} = \max(0, E_{current} - 110.0) \quad (14)$$

$$P_{excess} = -\frac{E_{excess}^2}{1000.0} \quad (15)$$

This penalty was directly added to the standard R_{energy} calculation. Due to time constraints, we were not able to run a new hyperparameter sweep on the hardware.

Another method we introduced to combat death spirals was to force the agent to learn how to recover while it was actively inside of one. Because the simulation was a "too perfect" environment that rarely entered death spirals organically, we introduced the `StateRandomizationWrapper`,

TABLE V
HYPERPARAMETER COMPARISON: TOP HARDWARE PERFORMING Q-LEARNING MODELS (v2)

Parameter	Model 4 (Trial 154)	Model 5 (Trial 249)
	<i>Agent Architecture</i>	
RBf Width (σ)	0.266	0.268
Learning Rate (α)	0.023	0.032
Discount Factor (γ)	0.970	0.971
Exploration Decay (ϵ_{decay})	0.983	0.990
	<i>Reward Function Weights</i>	
	Energy Penalty (w_{energy})	1.007
	Position Reward ($w_{position}$)	1.363
	Balance Bonus ($w_{balance}$)	1.388
	Stability Penalty (w_{stab})	0.005
		0.475
		2.407
		1.951
		0.132

which forced the model to start inside of a 40 rad/s death spiral for 20% of the episodes.

We heard from another group that transfer learning had worked well for them, so we gave it a try. We fine-tuned the v2 models for 600 episodes using the new excess energy penalty and the 20% chance of starting in a death spiral. However, we found out that this didn't work. All final reward values were under 500, while previous successful models scored anywhere between 2000-3000. We decided not to test these v3 models on the physical hardware.

8) *v4 models*: Instead of using transfer learning, we trained the v4 models from scratch using the 20% death spiral starts. Because the model now had to learn two entirely different scenarios (swing-up from rest and recovery from a death spiral), we increased the training length to 1200 episodes. We reused the hyperparameter configurations we had found during the v2 sweep.

We tested these models on the physical hardware. However, none of these models performed well overall. One of the models was able to successfully recover from a death spiral once (See Figure 4), but the overall balancing performance was degraded.

We think the v4 models failed because the way our reward

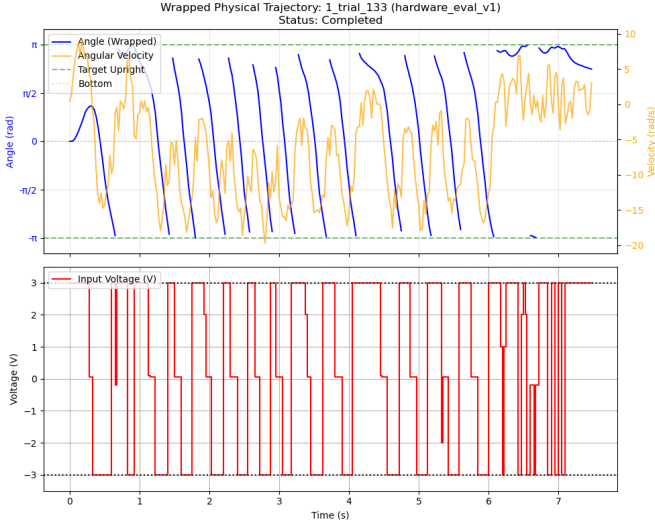


Fig. 4. First hardware test of model 1 of batch v4

function is structured with the energy reward is that it drives the models to increase ω whenever it is at the bottom half of the circle. However due to the input delay we measured, which is 2 timesteps, whenever the model tries to do a swing up, when it realizes it has reached the top and it wants to break, it takes 2 timesteps before the force is applied. Thus the model overshoots. Once it realizes that it has already missed the top it tries to compensate the position penalty with the energy reward. Thus accelerating again and making the catch the next time even harder due to the extra angular velocity.

We should also have introduced the massive uncertainty of the ω sensor and the motor input delay directly into the active training episodes, rather than just applying them during the evaluation phase to score the final policies. Forcing the agent to actively learn how to overcome these delays step-by-step during training would have produced models inherently more robust to the actual physical setup, ensuring the sim-to-real gap was significantly decreased.

C. Actor-Critic or Model Internalization Method

In addition to the Q-learning based controller, a continuous-action actor-critic controller was developed for the unbalanced disc swing-up problem. While the Q-learning approach discretizes the motor voltage into a finite set of actions, the physical actuator of the system is inherently continuous. Therefore, an actor-critic framework was investigated to directly learn a continuous control policy.

The implemented controller follows a Deep Deterministic Policy Gradient (DDPG)-style architecture. The policy network, or actor, maps the observed system state directly to a motor voltage command. To obtain a compact and continuous state representation, the input features were defined as

$$s = \left[\sin(\theta), \cos(\theta), \frac{\omega}{20}, \frac{e_{\text{top}}}{\pi} \right],$$

where θ is the measured disc angle, ω is the angular velocity, and e_{top} is the angular error with respect to the upright

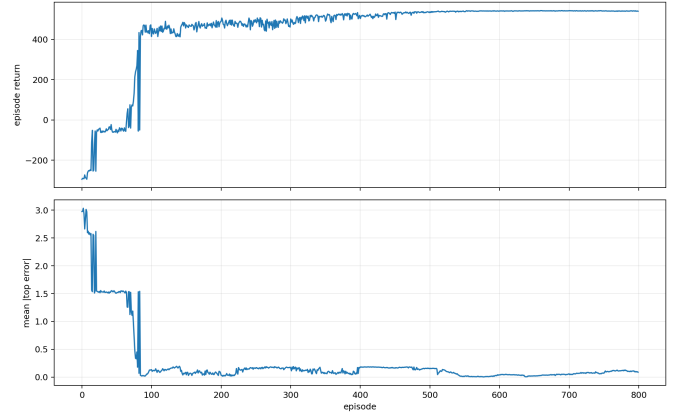


Fig. 5. Training progress of the actor-critic controller in simulation. The episode return increases during training, while the mean top-position error decreases, indicating that the policy learns both swing-up and stabilization.

equilibrium. The use of sine and cosine avoids discontinuities caused by angular wrap-around at $\pm\pi$.

The actor network consists of fully connected layers with ReLU activations, followed by a final hyperbolic tangent output layer. The final network output is multiplied by the actuator limit, producing a continuous voltage command

$$u \in [-3, 3] \text{ V},$$

which matches the physical voltage constraints of the setup. Unlike RBF Q-Learning, no action discretization is required, resulting in smoother control actions and a more natural interaction with the system dynamics.

Training was performed entirely in simulation for 800 episodes. During training, the episode return increased from negative values at the beginning to approximately 541–542 after convergence, while the mean upright error decreased to below 0.1 rad. This indicates that the controller learned both the swing-up phase and the subsequent stabilization phase.

As shown in Fig. 5, the actor-critic policy converged after approximately 100 episodes. The return increased sharply during the early training phase and then gradually stabilized, while the top-position error decreased close to zero. The remaining fluctuations are mainly caused by the exploration noise used during training.

The final policy was then tested in simulation without exploration noise. As shown in Fig. 6, the disc is swung up from the stable bottom equilibrium and brought close to the upright position $\theta = \pi$. After the swing-up transient, the top-position error converges close to zero and the angular velocity settles near zero. The control input remains within the allowed voltage range of ± 3 V throughout the evaluation. These results confirm that the continuous-action actor-critic policy successfully solves the swing-up and stabilization task in simulation.

To investigate practical applicability, the trained policy was subsequently deployed on the physical setup. For safety, a dedicated evaluation framework was implemented with voltage

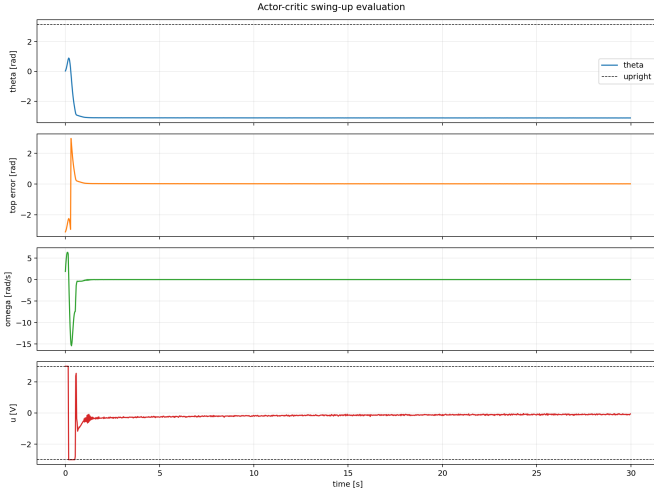


Fig. 6. Simulation evaluation of the trained actor-critic policy. The disc is swung up to the upright position, the angular velocity settles near zero, and the input voltage remains within the allowed range.

limits, rate limits, angular-velocity safety checks, and logging functionality. The framework recorded sensor measurements, corrected observations, policy outputs, final commanded voltages, and derived quantities at every control step, enabling detailed post-experiment analysis.

Although the controller achieved reliable performance in simulation, direct deployment on the real hardware was unsuccessful. Instead of stabilizing around the upright equilibrium, the disc continuously rotated in one direction and failed to settle. Inspection of the logged measurements revealed that the measured angular velocity often remained between approximately -2 and -5 rad/s even when the disc appeared physically stationary. Such behaviour suggests the presence of sensor bias or calibration errors.

As shown in Fig. 7, the hardware test did not reproduce the simulated swing-up behaviour. The measured angle accumulated over many rotations, while the angular velocity remained large and mostly negative. The commanded voltage also repeatedly reached the saturation limits, indicating that the learned policy was unable to recover once the real system deviated from the simulated behaviour.

The hardware test showed a clear sim-to-real gap, mainly due to sensor bias, input delay, and differences in the real motor response. Since the actor-critic policy was trained only in simulation, it was not robust enough to handle these effects.

Overall, the hardware test shows that the actor-critic policy did not transfer directly from simulation to the real setup. While the simulation result confirms that the learned policy can solve the swing-up and stabilization task under the simulated dynamics, the real experiment exposed important modelling and calibration differences. In particular, the observed angular-velocity bias and possible sign or actuator mismatches mean that additional calibration is required before the policy can be reliably tested on hardware. Future work should therefore focus on sensor calibration, actuator identification, and

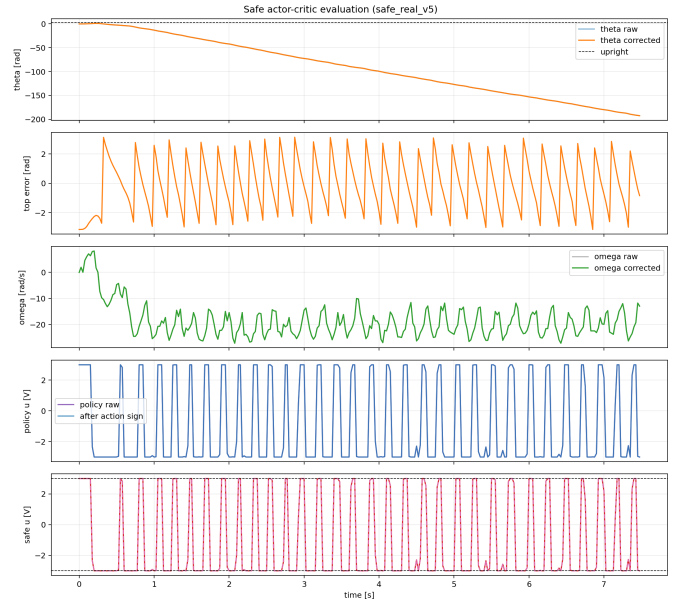


Fig. 7. Hardware evaluation of the actor-critic policy. Unlike the simulation result, the real disc did not stabilize near the upright position. The angle accumulated over multiple rotations and the angular velocity remained large, indicating continuous rotation caused by sim-to-real mismatch.

training with noise, delay, and parameter variations to reduce the sim-to-real gap.

V. SINGLE POLICY FOR SWING-UP AND REFERENCE TRACKING

Reference tracking was not implemented due to time constraints.

VI. DISCUSSION

Both GP and ANN outperform the linear baseline on the two tasks, as shown in Table IV, which confirms that the disc dynamics are dominated by the nonlinearity that a linear model cannot capture. On prediction the linear RMSE of 0.00665 rad is halved by both the GP-NARX (0.00340 rad) and the NARX ANN (0.00311 rad). The difference is far more evident on simulation, where the linear model degrades to 0.255 rad while both nonlinear models stay near 0.04 rad, which is an improvement of roughly a factor of six. This gap is expected: errors accumulate in free-run, so a structurally inadequate model is penalised much more heavily under simulation than under one-step prediction.

Q-Learning was more developed despite the lack of smooth control input it managed to balance the disk a couple of times. This was not managed by the Actor critic model. The actor critic model was not updated after the first hardware tests. Development was focused on the Q-Learning part. The real issue with the setup was the large sim to real gap. In the pristine simulation environment everything worked out well, however when applying these methods to the real setup they failed horribly. This was mainly due to motor input delay, the omega sensor having a wide variance and quite some bias. And

last but not least, the coefficients were different, like friction, motor output etc. Significant performance was denied by the decision to only evaluate the models by using the real setup quirks and not train them while these quirks were in effect.

VII. CONCLUSION

Both the GP-NARX and the NARX ANN capture the disc dynamics far better than the linear baseline and reach almost identical accuracy on prediction and simulation, so neither holds a clear advantage in fit on this system.

Despite the discontinuous action space of RBF Q-Learning we managed to create a model which could balance in 2/6 cases. This was the best performing model we could obtain. The actor-critic worked in simulation, but sensor bias, input delay, and model mismatch prevented successful hardware transfer within the available time.

AI TOOL DISCLOSURE

We used Gemini to plan out how the code would look like and to deliberate which options were the best, for instance the decision to choose RBF Q-Learning over the other Q-Learning methods was guided by a conversation with Gemini. It was used to display the facts and we made the decision ourselves. It was also used to inform us what sections of the rapport I should be clarified. This was then manually clarified. It was also used to find some spelling mistakes that were made during the rapport writing.

REFERENCES

APPENDIX A

ADDITIONAL RESULTS

Extra plots, hyperparameters, reward functions, or implementation details.

TABLE VI
OPTUNA HYPERPARAMETER TUNING SEARCH SPACE (EXPLORATORY VS. FOCUSED)

Parameter	Exploratory Range	Focused Range
<i>Agent Architecture</i>		
RBF Bins (n_{bins})	[5, 12] (int)	10 (constant)
RBF Width (σ)	[0.2, 1.5]	[0.20, 0.31]
Learning Rate (α)	[10^{-4} , 0.05] (log)	[0.010, 0.040] (log)
Discount Factor (γ)	[0.95, 0.999]	[0.95, 0.999]
Exploration Decay (ϵ_{decay})	[0.98, 0.999]	[0.98, 0.999]
<i>Reward Function Weights</i>		
Energy Penalty (w_{energy})	[0.1, 2.0]	[0.1, 2.0]
Position Reward ($w_{position}$)	[1.0, 5.0]	[1.0, 5.0]
Balance Bonus ($w_{balance}$)	[0.0, 4.0]	[0.0, 4.0]
Stability Penalty (w_{stab})	[0.0, 3.0]	[0.0, 3.0]